Inter-IIT Tech Meet 15.0

Selection Bootcamp – Speech (TTS)

Lightweight Hindi–Hinglish Voice Cloning · 5 days · free T4 GPU

Submission

A single file `<rollno>_tts_bootcamp.zip`, at most 100 MB, containing:

`code/`	notebook(s) or scripts; `README.md` with the exact commands that reproduce every submitted file, and a note on any AI coding assistant you used
`predictions/`	audio folders and files listed under Evaluation
`idea.md`	your idea card: failure, method, prediction, result ($\leq$ 200 words in total)
`results.json`	filled-in copy of the template: seed, settings, method name, validation numbers, targeted failure and category, speed-up
`report.md`	answers to the fixed questions (Q1–Q3), $\leq$ 75 words each

Run `check_format.py` on your zip before every upload. **A file that fails it scores 0.**

Problem Statement – Zero-Shot Voice Cloning for Hindi and Hinglish

Problem Statement

Voice assistants in India must read messages that mix Hindi and English and are full of numbers, dates and rupee amounts, in a voice the customer chooses. Zero-shot text-to-speech (TTS) models can clone a voice from a ten-second clip, but they fail in predictable ways: they drop or repeat words, misread numbers, mangle English words inside Hindi sentences, and lose the speaker or the rhythm on long text. Fixes come in two kinds, and they behave differently:

- **Text normalization** is cheap. It fixes numbers, dates and abbreviations, but not how English words are pronounced.
- **Changing how the model is used** (the prompt, the decoding settings, how long text is split, choosing among candidates) can fix more, but some of it costs speed.

Your job is to take a pretrained zero-shot TTS model, measure where it fails, write a text frontend for it, and then invent and test one method of your own that reduces one kind of failure. Nothing is trained. Graders reward a well-argued idea tested honestly; a clear negative result with a good diagnosis counts as fully as a positive one. The bootcamp takes about 10 hours over five days. Each task is scored separately, so submit whatever you finish.

The Baseline Model

The baseline is **IndicF5** (AI4Bharat): a 330M-parameter flow-matching model fine-tuned from F5-TTS on about 1,400 hours of Indian-language speech. To clone a voice it needs a reference clip **and the transcript of that clip**; `data/prompts.csv` gives you both. The wrapper exposes `synthesize(text,`

`prompt_wav, prompt_text, seed, **overrides)`, which returns 16 kHz audio, and pins everything else in `baseline/config.yaml`: model revision, decoding settings and seed. Running it unchanged reproduces `data/baseline_val/`.

> **Day-1 setup**
> The weights are gated on Hugging Face. Create a free account, accept the terms at `huggingface.co/ai4bharat/IndicF5`, and add your token to the notebook as `HF_TOKEN`. Approval is usually immediate, but do it before you need the GPU.

Rules

- **Model:** IndicF5 as pinned in `baseline/`. No other TTS model, no closed or paid speech service, no generative LLM, and no other model anywhere on the path from text to audio (transliteration, grapheme-to-phoneme and text-normalization models included). Open-weight recognizers and speaker models may be used only as **verifiers**, to score outputs or choose among them.
- **Data:** only the files in the kit. Hand-written rules and word lists are allowed, but build them from validation items only. You may apply them to evaluation items; you may not extend them by reading evaluation text.
- **What your method may change:** the text going into the model; the reference clip and its transcript (trimming, choosing a segment, correcting the transcript); decoding settings; how long text is split and joined; and the choice among **at most 4 candidates** per item. Not model weights, and no added models other than verifiers.
- **Audio post-processing:** only trimming leading and trailing silence, loudness normalization, and joining the pieces of a split item with a pause or crossfade of at most 0.3 s. No speed or pitch change, filtering or denoising.
- **Tuning:** validation items only. Never choose among evaluation outputs by listening; any choice among candidates must be made by your code.
- **Compute:** any free hardware (Kaggle gives about 30 free T4 hours a week; Colab also works). Task 1 and the text part of Task 2 need no GPU.
- **Individual work:** ask questions only in the public channel; do not share code, audio or results. AI coding assistants are allowed if disclosed in your README; `idea.md` and `report.md` must be your own words. The last upload before the deadline counts; late work is not accepted.

Tasks

Task 1 – Measure the Baseline (no GPU needed)

- Score the provided `data/baseline_val/` with `metrics.py` and record validation CER and speaker similarity (SIM) in `results.json`. They should match `data/baseline_published.json`.
- Listen to every item whose CER exceeds 0.20 and tag it with the **one** category that best describes its main error. You may also tag items below 0.20 if you hear a clear error.

Task 2 – Text Frontend

- Write your own rule-based `normalize(text)`, without any model, that rewrites text into Hindi words as spoken, in Devanagari: Indian-style numbers (lakh, crore), ₹ amounts, dates and common abbreviations.
- Leave English words written in Latin script unchanged; how they are pronounced is a Task 3 problem. Abbreviations such as SBI or KYC are the exception: spell them out.
- Submit the normalized text of all 64 items, then synthesize the 40 evaluation items with normalization on and everything else as in the baseline.

Task 3 – Your Idea

- Pick one failure category from your Task 1 tags and invent a method that reduces it. Tuning decoding settings and plain best-of-N reranking with a recognizer are **baselines, not ideas**: you may use them, but your method must do something beyond them.
- **Transliteration:** a lookup table that maps the English words in the validation items to Devanagari is also a baseline. Hand-written rules that convert English words you have *not* seen (spelling patterns, silent letters, Indian-English vowel sounds) can be part of an idea. Most English words in the evaluation items do not appear in the validation items, so only rules that generalize will help there. Either way it must be rule-based: no transliteration or pronunciation models.
- **Write the plan before you run it.** In `idea.md` state (1) the failure you target, with 2 item IDs from `baseline_val/`; (2) your hypothesis and method; (3) your prediction: which category improves, by roughly how much, and what could go wrong. Submit parts 1–3 on the Day-3 form by 23:59 on Day 3; its timestamp proves the prediction came first. Do not edit them afterwards.
- Run your method against the baseline on the same 24 validation items with the same seeds. If it builds on your Task 2 text, also submit `val_control/` (Task 2 text, baseline otherwise), so the idea's effect can be separated from normalization's.
- Add (4): the result, and what it taught you. In `results.json`, set `targeted_failure` (your Task 1 tag) and `targeted_category` (the item category your score is computed on).

Task 4 – Faster (optional bonus)

- Make your Task 3 system faster without raising its validation CER by more than 20%.
- Report the speed-up as baseline time ÷ your time to synthesize the 24 validation items, model loaded and one warm-up excluded, both measured on the same machine in the same session. `baseline/run_baseline.py` writes the baseline's timing file; put both in `code/`.

Expected Outcomes

- A measured baseline with a short taxonomy of its errors.
- A rule-based text frontend and its measured effect on the audio.
- One idea of your own, with a prediction written before the test and an honest account of the

result.

Dataset

Source: reference clips of 8 to 12 seconds from the Hindi test split of AI4Bharat's IndicVoices-R (CC BY 4.0), each from a different speaker, and sentences written by the organizers. There is no training data, because nothing is trained.

What's provided (in `tts_bootcamp_kit/`)

- `baseline/`: the wrapper with its pinned settings (`tts.py`, `config.yaml`, `requirements.txt`), a batch runner that also times runs (`run_baseline.py`), and a Colab notebook
- `data/val_items.csv`: 24 validation items (`id`, `category`, `text`, `prompt_id`, `spoken`)
- `data/eval_items.csv`: 40 evaluation items (`id`, `category`, `text`, `prompt_id`); no `spoken` column and no reference audio
- `data/prompts/` and `data/prompts.csv`: 10 reference clips (24 kHz wav) with transcripts, from 4 validation speakers (V1–V4) and 6 evaluation speakers (E1–E6)
- `data/baseline_val/`: the baseline's audio for all 24 validation items; its validation CER and SIM are in `data/baseline_published.json`
- `metrics.py`: development recognizer and speaker model; scores a folder of validation audio for CER, SIM and sanity checks
- `results_template.json`: copy it to `results.json` and fill it in
- `check_format.py`: validates your submission's file names, formats and word limits

Categories. Every item is one of `hindi` (Devanagari only), `hinglish` (Hindi with 2–4 English words, in mixed script: Devanagari for Hindi, Latin for English), `hard` (numbers, dates, ₹ amounts, abbreviations) or `long` (60 to 100 words, mostly Hindi).

Set	Hindi	Hinglish	Hard	Long	Total
Validation	6	6	8	4	24
Evaluation	10	10	14	6	40

> **Note**
>
> `spoken` is what an item should sound like, written out in Devanagari words; CER is computed against it. Where an item has more than one acceptable reading (a train number read as one number or digit by digit), the readings are separated by `||` and the closest one counts. The final scoring recognizers and speaker model are **not** the ones in your kit, so do not over-fit to `metrics.py`.

Evaluation

Prediction files (in `predictions/`)

- `t1_val_errors.csv`: format `item_id,category`, one row per tagged item. Allowed categories: `skipped_words`, `repeated_words`, `number_error`, `english_word_error`, `speaker_mismatch`, `other`.
- `t2_normalized.csv`: format `id,split,text,normalized_text`, 64 rows (24 validation, 40 evaluation), `split` equal to `val` or `eval`.
- **Evaluation audio:** `t2_norm/` and `t3_idea/` (plus `t4_fast/` for the bonus), each holding exactly 40 files named `<item_id>.wav`.
- **Validation audio:** `val_idea/` (24 files), plus `val_control/` (24 files) if your idea builds on Task 2. These make your report checkable.
- **Audio format:** mono, 16 kHz, 16-bit PCM wav, between 0.5 and 60 seconds, and nothing else in the folder. A missing, silent or over-long file is a failed item.

Scoring

File	Scored by
`t1_val_errors.csv`	Each tagged item must be a real error, checked against the scored CER and by ear, with a sensible category
`t2_normalized.csv`	Completeness and sanity: no digits, ₹ or % left; no Latin letters except English words that were in Latin script in the input
`t2_norm/`	CER on hard items against the baseline (primary); CER on each other category must not rise by more than 5%
`t3_idea/` + `val_idea/`	Relative CER reduction on your `targeted_category` (primary). Guards: overall CER at most 10% worse and SIM at most 0.03 lower than the baseline's
`idea.md`	Two seniors: grounding, testability, originality, insight
`t4_fast/` (optional)	CER at most 1.2 times that of `t3_idea/`; speed-up from `results.json`
`results.json`	Reported validation numbers match, within 0.01, what `metrics.py` gives on the validation audio you submitted

How metrics are computed

- **CER:** the mean over several recognizers not in your kit, computed against the `spoken` reference. Transcripts are produced as Hindi words in Devanagari (English words transliterated, numbers as words); both sides are stripped of punctuation and common spelling variants are folded together. A failed item counts as CER 1.
- **SIM:** cosine similarity between the output and its prompt clip, from a speaker-verification model not in your kit.

- **Sanity checks:** speaking rate, share of silence and clipping. Items outside natural ranges count as failed. A run with more than 10% failed items is not scored.
- **Speed (Task 4):** a ratio to the baseline on your own machine in the same session, so your hardware does not matter.

Report Questions

(`report.md`, ≤ 75 words each; all analysis on **validation** items. Start each answer on a line beginning `Q1`, `Q2`, `Q3`, and do not paste the question.)

Q1. Give two validation item IDs where your idea system has lower CER than the baseline and two where it has higher, and explain why. If none is higher, give the two with the smallest gain. `baseline_val/` *and your* `val_idea/` *make this checkable.*

Q2. What did the result teach you about why the model fails? Name one thing you would now do differently.

Q3. What single change would you try next, and why?